

Overlay of Subdivisions

1 Doubly Connected Edge Lists

- walking counterclockwise along edges enclosing a face

2 Computing Overlays of Subdivisions

- problem statement
- compute new half edges and update the faces
- cost analysis

3 Boolean Operations

- union, intersection, difference

MCS 481 Lecture 6
Computational Geometry
Jan Verschelde, 4 September 2026

Overlay of Subdivisions

1 Doubly Connected Edge Lists

- walking counterclockwise along edges enclosing a face

2 Computing Overlays of Subdivisions

- problem statement
- compute new half edges and update the faces
- cost analysis

3 Boolean Operations

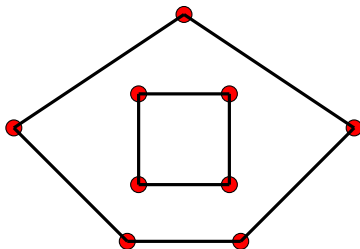
- union, intersection, difference

planar subdivisions

A planar subdivision is an embedding of a graph in the plane:

- 1 a vertex is a node in the graph,
- 2 an edge connects two nodes in the graph,
- 3 a face is enclosed by edges and vertices.

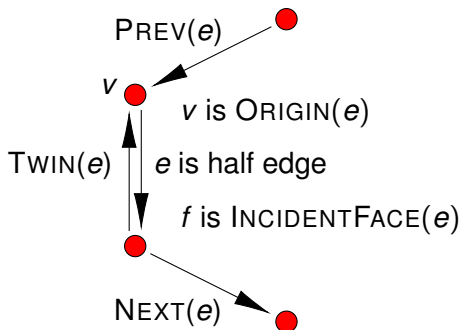
An example of 9 vertices, 9 edges, and 3 faces:



Check Euler's formula!

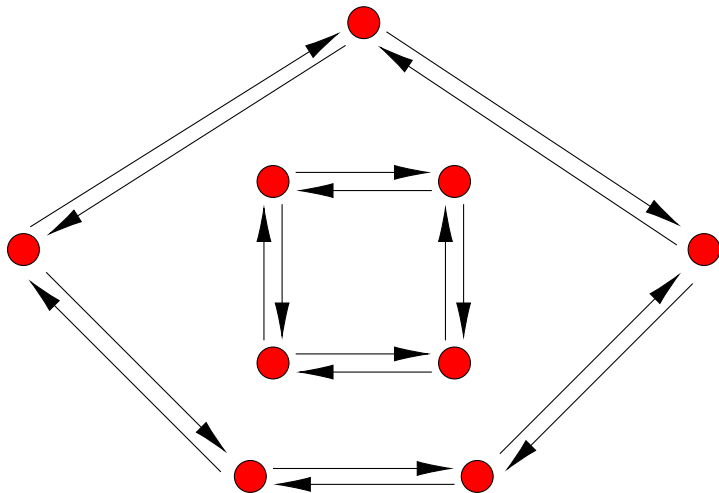
walking counterclockwise along edges

An edge record stores five items for a half edge e , e.g.:



The half edges are oriented so that we walk in counterclockwise fashion around the incident face, the face enclosed following $\text{NEXT}(e)$.

walking around the faces



Overlay of Subdivisions

1 Doubly Connected Edge Lists

- walking counterclockwise along edges enclosing a face

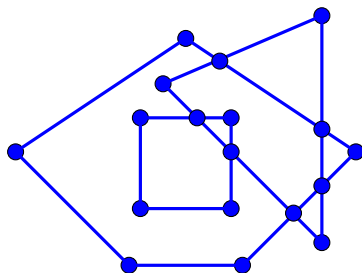
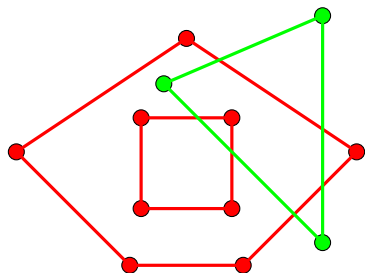
2 Computing Overlays of Subdivisions

- problem statement**
- compute new half edges and update the faces
- cost analysis

3 Boolean Operations

- union, intersection, difference

problem statement



Input: two doubly connected edge lists S_1 and S_2 .

Output: $\mathcal{O}(S_1, S_2)$, the overlay as a doubly connected edge list.

Exercise 1: Draw the doubly connected edge list for $\mathcal{O}(S_1, S_2)$.

the algorithm MAPOVERLAY

Algorithm MAPOVERLAY(S_1, S_2)

Input: two doubly connected edge lists S_1 and S_2 .

Output: $\mathcal{O}(S_1, S_2)$, the overlay as a doubly connected edge list.

- 1 Let $\mathcal{D}$ be the edge lists of S_1 and S_2 .
- 2 Let $P = \text{FINDINTERSECTIONS}(\mathcal{D})$, apply line sweep algorithm.
- 3 For each $p \in P$ do
 - 4 compute new half edges at p
 - 5 connect the NEXT() and PREV() pointers at p
- 6 Use $\mathcal{D}$ to update the faces.

Overlay of Subdivisions

1 Doubly Connected Edge Lists

- walking counterclockwise along edges enclosing a face

2 Computing Overlays of Subdivisions

- problem statement
- compute new half edges and update the faces**
- cost analysis

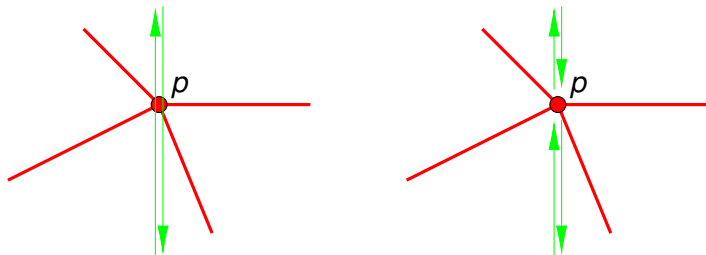
3 Boolean Operations

- union, intersection, difference

compute new half edges at intersection points

- 4 Compute first the new half edges, and
- 5 then connect the NEXT() and PREV() pointers.

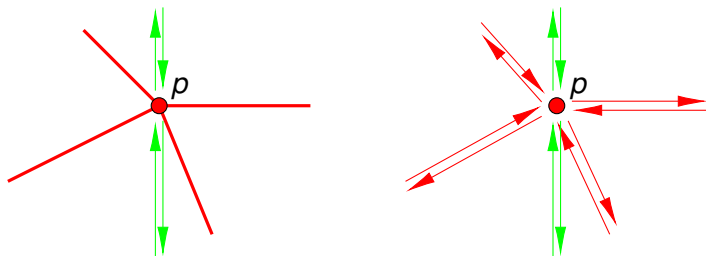
Consider the case where p is already a vertex of S_1 , where p lies on an edge e of S_2 :



The edge e is splitted at p :

- 1 The shortened edges get new half edges as twins.
- 2 The new half edge records have p as their ORIGIN() item.

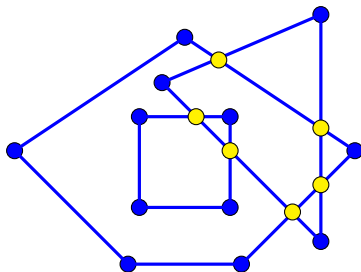
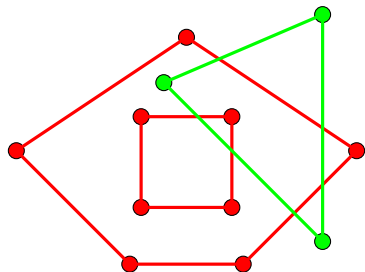
connect the NEXT() and PREV() pointers



- 1 Every shortened edge e has its NEXT() changed to the first counterclockwise half edge with ORIGIN() at p .
- 2 That first counterclockwise half edge with ORIGIN() at p will have its PREV() set to the shortened edge e .
- 3 The new half edges copy their NEXT() from the old half edges.
- 4 The PREV() for the new half edge e is the first clockwise edge that has its destination at p .
- 5 That first clockwise half edge will set its NEXT() to e .

an exercise

Exercise 2: Consider the six new intersection points, shown in yellow at the right in the picture below.



Choose the leftmost new intersection point and describe in sufficient detail the setting of the `NEXT()` and `PREV()` pointers for all half edges involved in your chosen intersection point.

update the face records

After

- 1 the computation of all intersection points,
 - 2 the computation of the new half edges, and
 - 3 the connectivity of the doubly connected edge lists, we have
- to determine for each face its outer component and inner components.

Definition (face record)

A *face record* stores two items:

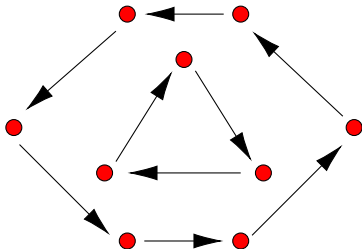
- 1 INNERCOMPONENTS: half edges, one for each hole,
- 2 OUTERCOMPONENT: a half edge to its outer boundary.

two observations about the faces

Invariant of doubly connected edge lists:

the enclosing face is always at the left

when walking on the edge list, around the face.



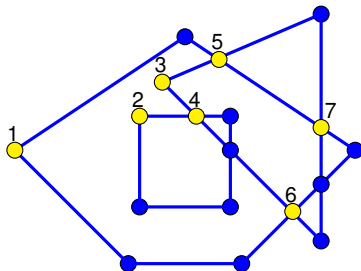
Observe the angle at a left turn:

- If the angle $< \pi$, then we walk on the outer boundary.
- If the angle $> \pi$, then we walk on the inner boundary.

definition of a graph

We define a graph G linking the highest leftmost vertices in each cycle of edges enclosing a face.

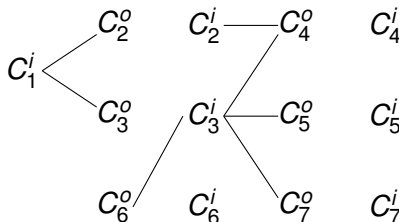
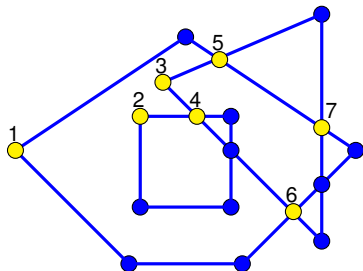
In the example below, we have seven faces, each enclosed by a cycle, the highest leftmost vertex in every enclosing cycle is shown in yellow.



Observe: the cycle at the inner boundary for the first face is an outer boundary cycle for faces 2, 3, 4, 5, and 6, in counterclockwise order starting at yellow vertex 1, visiting 6, 2, 4, 3, and 5.

connected components in the graph

For a face f , denote C_f^i is the inner boundary cycle for face f , and C_f^o is the outer boundary cycle for face f .



Lemma (connected components in the graph)

Each connected component in the graph corresponds exactly to the set of cycles incident to one face.

using the graph to update the face records

In MAPOVERLAY, we have the statement

- ⑥ Use $\mathcal{D}$ to update the faces.

which can now be made more precise, replacing the statement by

- ⑥ Make the graph G with nodes corresponding to boundary cycles and edges connecting each hole cycle to the left of its highest leftmost vertex.
- ⑦ Compute the connected components of G .
- ⑧ For each connected component in G do
- ⑨ make face records for each face bounded by cycles.

Exercise 3: For the example on the previous slide, define all face records, illustrating the use of the graph.

Overlay of Subdivisions

1 Doubly Connected Edge Lists

- walking counterclockwise along edges enclosing a face

2 Computing Overlays of Subdivisions

- problem statement
- compute new half edges and update the faces
- cost analysis**

3 Boolean Operations

- union, intersection, difference

cost analysis

Theorem (cost of map overlay)

The cost of MAPOVERLAY for two subdivisions S_1 and S_2 is $O((n + k) \log(n))$ where

- *n is the number of vertices in S_1 and S_2 ; and*
- *k is the complexity of the overlay $\mathcal{O}(S_1, S_2)$.*

Step 2 in MAPOVERLAY applies Algorithm FINDINTERSECTIONS, which has cost $O((n + I) \log(n))$, where n is the number of segments and I the number of intersection points.

Step 4 and 5 of MAPOVERLAY, the computation of the new half edges at the intersection points and the connectivity computation have a cost proportional to the degree of the vertices.

Overlay of Subdivisions

1

Doubly Connected Edge Lists

- walking counterclockwise along edges enclosing a face

2

Computing Overlays of Subdivisions

- problem statement
- compute new half edges and update the faces
- cost analysis

3

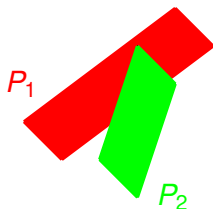
Boolean Operations

- union, intersection, difference

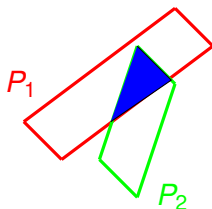
union, intersection, difference

Given two polygons P_1 and P_2 , we extract those faces

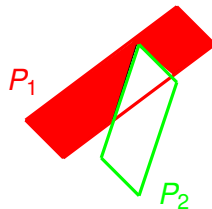
- labeled with P_1 or P_2 : union, $P_1 \cup P_2$,
- labeled with P_1 and P_2 : intersection, $P_1 \cap P_2$,
- labeled with P_1 and not P_2 : difference, $P_1 \setminus P_2$.



$$P_1 \cup P_2$$



$$P_1 \cap P_2$$



$$P_1 \setminus P_2$$

application of the map overlay theorem

Corollary (application of the map overlay theorem)

Boolean operations can be done with cost $O((n + k) \log(n))$ where

- *n is the total number of vertices; and*
- *k is the complexity of the output.*

exercises

We closed the second chapter in the textbook.

Consider the following activities, listed below.

- 1 Install CGAL on your computer (if not already done so).
Run the examples in the `/examples/Arrangement_2` folder of CGAL and browse the documentation.
- 2 Browse the documentation for the half edge data structures:
<https://doc.cgal.org/latest/HalfedgeDS/index.html>
- 3 Explore the Python interface to CGAL, and in particular the half edge data structures exported to Python.
- 4 Write the solutions to exercises 1, 2, and 3.
- 5 Consider the exercises 7,8,9 in the textbook.